



```
In [1]: import numpy as np
```

```
In [2]: def calculate_distance(city1, city2):  
        return np.linalg.norm(np.array(city1) - np.array(city2))
```

```
In [3]: def nearest_neighbor_algorithm(city_coordinates):  
        num_cities = len(city_coordinates)  
        unvisited_cities = set(range(num_cities))  
        tour = []  
        # Start from a random city  
        current_city = np.random.choice(list(unvisited_cities))  
        unvisited_cities.remove(current_city)  
        tour.append(current_city)  
        while unvisited_cities:  
            nearest_city = min(unvisited_cities,  
                              key=lambda city: calculate_distance(city_coordinates[city],  
                                                                    city_coordinates[current_city]))  
            unvisited_cities.remove(nearest_city)  
            tour.append(nearest_city)  
            current_city = nearest_city  
        return tour
```

```
In [4]: def calculate_total_distance(tour, city_coordinates):  
        total_distance = 0  
        for i in range(len(tour)-1):  
            total_distance += calculate_distance(city_coordinates[tour[i]], city_coordinates[tour[i+1]])  
        return total_distance
```

```
In [5]: # Example usage  
        city_coordinates = [(0, 0), (1, 2), (3, 1), (5, 4), (2, 6)]  
        tour = nearest_neighbor_algorithm(city_coordinates)  
        total_distance = calculate_total_distance(tour, city_coordinates)  
  
        print("Optimal tour:", tour)  
        print("Total distance:", total_distance)
```

Optimal tour: [np.int64(1), 0, 2, 3, 4]

Total distance: 16.73255381421381